

文档编号: WikiBridge-SDS-1.1

WikiBridge

软件设计规格说明书

零代码本地知识库 API 分享与 OpenCode/MCP 消费平台

项目名称: WikiBridge

文档版本: V1.1

编写人员: 项目组

适用阶段: 课程设计原型与系统设计评审

日期: 2026 年 6 月 26 日

文档变更历史记录

序号	变更日期	变更人员	变更内容详情描述	版本
1	2026 年 6 月 1 日	项目组	启动系统设计调研，明确本地知识库、内网穿透、桌面端和对话式知识消费的总体设计目标。	V0.1
2	2026 年 6 月 10 日	项目组	补充 B 端、S 端、C 端初始职责划分，形成体系结构、用户界面、用例和部署设计草案。	V0.5
3	2026 年 6 月 18 日	项目组	基于现有项目简介、汇报材料、架构图与用例图，撰写 WikiBridge 软件设计规格说明书主体内容。	V1.0
4	2026 年 6 月 25 日	项目组	根据联调结果完成 MCP 架构调整：淘汰 B 端发布完整 OpenCode 方案，改为 B 端仅发布 LLM-Wiki API，S 端提供 BearFRP/frps 控制面，C 端本地运行 OpenCode 并通过 MCP 消费远程知识库。	V1.1
5	2026 年 6 月 26 日	项目组	整理章节、图表、术语和参考资料，按课程提交格式生成软件设计说明书 PDF。	V1.1

目录

文档变更历史记录 I

1 引言 1

1.1 编写目的 1

1.2 读者对象 1

1.3 软件项目概述 1

1.4 文档概述 2

1.5 定义 2

1.6 参考资料 3

2 软件设计约束 3

2.1 软件设计目标和原则 3

2.2 软件设计的约束和限制 4

3 软件设计 4

3.1 软件体系结构设计 4

3.1.1 B 端子系统 5

3.1.2 S 端子系统 5

3.1.3 C 端子系统 5

3.2 用户界面设计 5

3.3 用例设计 6

3.3.1 构建本地知识库用例 7

3.3.2 发布 LLM-Wiki API 用例 7

3.3.3 添加远程知识库用例 7

3.3.4 启动本地 OpenCode 与注册 MCP 用例 7

3.3.5 对话式读取与搜索用例 7

3.4 核心流程设计 8

3.5 类设计和构件设计 8

3.5.1 关键类关系 9

3.5.2 关键方法设计 9

3.6 数据设计 10

3.6.1	B 端数据结构	10
3.6.2	C 端本地配置数据	11
3.6.3	S 端控制面数据表	11
3.6.4	数据安全设计	12
3.7	部署设计	12
3.7.1	B 端部署	12
3.7.2	S 端部署	12
3.7.3	C 端部署	12
3.8	异常处理与可靠性设计	13
3.9	测试设计	13
结论		14

1、引言

1.1 编写目的

软件设计过程包括体系结构设计、用户界面设计、用例设计、构件设计、类设计、数据设计和部署设计。本文档以软件设计规格说明书的形式，对 WikiBridge 零代码本地知识库 API 分享与 OpenCode/MCP 消费平台的设计方案进行系统描述，形成可用于评审、开发分工、测试设计和后续迭代的设计基线。

本文档重点说明系统如何在 B 端生成和持有本地知识库，如何通过 BearFRP/frp 将 LLM-Wiki API 发布为受控公网入口，以及 C 端如何在本地运行 OpenCode 并通过 `llm-wiki-*` MCP 工具访问远程知识库。本文档以联调后确定的新架构为准：B 端不再直接发布完整 OpenCode 页面，也不把 C 端访客的 Agent 行为运行在 B 端环境中。

1.2 读者对象

本文档面向以下读者：

- 项目评审人员和课程设计指导教师，用于评估系统设计的完整性、合理性和可实现性。
- 软件设计人员和开发人员，用于理解系统边界、模块职责、关键流程和数据结构。
- 测试人员和质量保证人员，用于设计功能测试、异常测试和端到端验收场景。
- 配置管理人员和运维人员，用于理解本地端、服务器端和公网访问端的部署关系。

1.3 软件项目概述

项目名称为 WikiBridge，定位为零代码本地知识库 API 分享与受控 Agent 化消费平台。项目面向教师、学生、研究者、小团队和其他低技术门槛用户，解决个人资料分散、AI 知识库部署复杂、本地知识库难以安全分享、Agent 能力公网暴露边界不清的问题。

系统的核心思想是把知识库生成、发布和消费拆分到三个明确边界中。B 端知识库拥有者在本地运行 LLM-Wiki，保留原始资料、Wiki 页面和知识库 API；B 端只通过 frpc/frps 发布 LLM-Wiki API，而不是发布完整 OpenCode Agent。S 端运行 BearFRP 控制面和 frps，负责账号、鉴权、子域名或端口分配、隧道状态和公网入口转发。C 端知识消费者在自己的设备上运行 WikiBridge Desktop 与本地 OpenCode，保存自己的模型供应商、模型名和 API Key，并通过本地 MCP server 将 B 端分享的 LLM-Wiki API 注册为 `llm-wiki-*` 工具。

系统划分为三个端：

- B 端：知识库拥有者和发布者所在环境，负责本地资料整理、LLM-Wiki 构建、LLM-Wiki API 服务和 frpc 发布。

- S 端：公网服务器端，负责 BearFRP 控制面、frps、HTTP vhost、发布入口、账号鉴权和隧道状态。
- C 端：知识消费者所在环境，负责本地 OpenCode、MCP 注册、模型配置、远程知识库连接和对话式知识消费。

1.4 文档概述

本文档共分为三大部分。第一部分为引言，说明编写目的、读者对象、项目概述、术语定义和参考资料。第二部分为软件设计约束，说明设计目标、设计原则、运行约束、安全约束和技术约束。第三部分为软件设计，分别描述体系结构、用户界面、用例、类与构件、数据和部署。

1.5 定义

术语	定义
WikiBridge	本项目名称，表示连接本地知识库、受控公网入口和本地 Agent 消费端的桥接平台。
B 端	知识库所有者/发布者所在环境，保存本地资料、Wiki 内容和 LLM-Wiki API。
S 端	公网服务器端，运行 BearFRP 控制面、frps 和 HTTP vhost。
C 端	知识消费者所在环境，本地运行 WikiBridge Desktop、OpenCode 和 MCP server。
LLM-Wiki	本地知识库编译与 API 服务，提供健康检查、项目列表、文件读取和搜索等接口。
OpenCode	C 端本地对话式知识消费入口，负责模型调用、KB Chat Session 和工具调用。
MCP	Model Context Protocol，用于将远程 LLM-Wiki API 注册为 OpenCode 可调用工具。
BearFRP	本项目中的 frp 控制面，负责账号、代理、流量、子域名/端口和发布入口。
frp / frpc / frps	开源内网穿透工具及其客户端/服务端；B 端运行 frpc，S 端运行 frps。
KB 模式	OpenCode 的知识库模式，限制文件访问和工具能力范围。

API 分享	B 端通过 BearFRP 分享 LLM-Wiki API，而不是分享完整 OpenCode 页面。
控制面	S 端负责的账号、鉴权、隧道调度、状态记录和资源控制能力。
数据面	B 端知识内容、C 端模型配置、MCP 工具调用和远程 API 响应等业务数据流。

1.6 参考资料

1. README.deploy.md: WikiBridge 部署和 BearFRP 发布说明。
2. desktop/HANDOFF.md: WikiBridge Desktop 交接说明。
3. /root/SE/doc/images/architecture/architecture_comparison.md: 新旧架构对比说明。
4. /root/SE/doc/软件计划任务书/SDS-图表素材.md: 软件设计说明书图表素材。
5. 软件设计规格说明书模板: 软件设计规格说明书-空巢老人看护系统.doc。

2、 软件设计约束

2.1 软件设计目标和原则

WikiBridge 的软件设计目标是在课程设计周期内形成可演示、可解释、可扩展的端到端原型设计。系统应让 B 端用户不需要手写 frp 配置即可发布本地 LLM-Wiki API，让 C 端用户只需粘贴远程知识库 API 地址、配置自己的模型 Key，即可在本地 OpenCode 中对远程知识库进行对话式消费。

系统设计遵循以下原则：

- 本地优先原则：B 端原始资料、Wiki 页面和 LLM-Wiki 数据保留在 B 端；C 端模型供应商、模型名和 API Key 保留在 C 端；S 端不保存知识正文和问答上下文。
- B/S/C 职责分离原则：B 端负责知识库生成和 API 分享，S 端负责 BearFRP 控制面和公网转发，C 端负责本地 OpenCode、MCP 注册和模型调用。
- Agent 边界收敛原则：不通过 frp 暴露 B 端完整 OpenCode；OpenCode Agent 能力运行在 C 端本地，且通过 KB 只读模式和 llm-wiki-* MCP 白名单收紧工具范围。
- 模块化与低耦合原则：LLM-Wiki API、BearFRP/frp、WikiBridge Desktop、OpenCode sidecar 和 MCP server 通过清晰接口协作，便于替换和扩展。

- 主动触发原则：发布 API、添加远程知识库、启动 OpenCode、注册 MCP 和连接远程知识库均由用户显式操作或确认。
- 可观测原则：LLM-Wiki 健康状态、frp 隧道状态、远程知识库可用性、OpenCode 启动状态和 MCP 注册状态均应在界面中可见。
- 可追踪原则：功能需求、用例、模块、数据结构和测试场景之间保持可追踪关系。

2.2 软件设计的约束和限制

约束类别	约束说明
运行环境	B 端运行 LLM-Wiki API 和 frpc；S 端运行 BearFRP back-end、frps 和控制面数据库；C 端运行 WikiBridge Desktop、OpenCode 和 MCP server。
网络条件	B 端通常处于内网或 NAT 环境，公网访问依赖 frpc 与 frps 建立隧道；HTTP 子域名模式依赖真实可解析的通配域名。
隐私边界	S 端不保存用户原始资料、Wiki 正文、向量数据或问答上下文；C 端模型 API Key 不上传到 S 端或 B 端。
安全约束	B 端不暴露完整 OpenCode；发布入口需要可回收、可鉴权、可限速；Token 和 API Key 不应明文长期暴露在界面或日志中。
能力约束	OpenCode KB 模式应阻断危险文件访问、终端执行和不受控工具能力；MCP 接入只允许项目内 llm-wiki-* 工具。
技术约束	当前阶段以原型验证为主，优先保证 B/S/C 主链路闭环；静态 Wiki 页面浏览可作为降级方案，不作为主消费路径。
课程周期	设计以可演示闭环为优先，先打通 B 端 API 发布、C 端远程知识库添加、OpenCode + MCP 对话，再逐步完善体验和边界场景。

3、 软件设计

3.1 软件体系结构设计

系统采用 B/S/C 三端分层架构。B 端负责知识库内容和 LLM-Wiki API，S 端负责 BearFRP 控制面和公网转发，C 端负责本地 OpenCode、MCP 注册和模型调用。总体架构如图 1 所示。

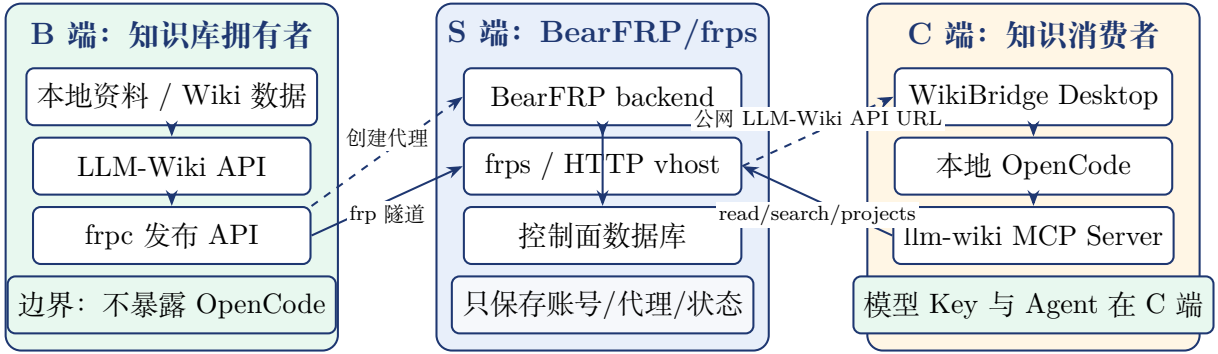


图 1: WikiBridge B/S/C 总体架构图

3.1.1 B 端子系统

B 端是知识库拥有者的核心操作环境，承担知识内容持有和 API 发布职责。其内部包含 LLM-Wiki 数据目录、LLM-Wiki API 服务、WikiBridge Desktop 发布控制界面和 frpc。B 端可以构建、浏览和管理本地知识库，但对外只发布 LLM-Wiki API，不发布完整 OpenCode 页面。

LLM-Wiki API 至少提供健康检查、项目列表、文件读取、搜索和元数据接口。frpc 仅将该 API 映射到 S 端公网入口，避免外部访客获得 B 端本机文件系统、模型 Key 或 OpenCode Agent 能力。

3.1.2 S 端子系统

S 端只处理平台控制面和转发面，不进入用户知识内容。BearFRP backend 负责用户、余额、代理、发布记录、脚本生成和在线状态；frps 负责接收 B 端 frpc 连接并根据 HTTP vhost 或 TCP 端口转发请求；控制面数据库保存账号、代理和隧道状态，不保存原始资料、Wiki 正文或问答上下文。

3.1.3 C 端子系统

C 端是知识消费者的本地 Agent 环境。WikiBridge Desktop 负责保存远程知识库 API 地址、检测健康状态、选择项目、启动本地 OpenCode 并注册 llm-wiki-* MCP。OpenCode 在 KB 模式下调用 MCP 工具读取或搜索远程知识库，模型供应商、模型名和 API Key 均保存在 C 端本地。

3.2 用户界面设计

WikiBridge Desktop 以 B 端发布和 C 端消费两类任务为核心，主要界面如表 4 所示。界面设计目标是让用户无需手写 frp 配置、MCP 配置和 OpenCode 启动命令，也能完成远程知识库发布和消费。

表 4: WikiBridge Desktop 主要界面设计

界面	主要元素	职责
欢迎/引导界面	项目简介、B/S/C 边界、本地优先说明、开始按钮	解释系统用途和隐私边界，引导用户进入工作台。
模型设置界面	模型供应商、模型名、API Key、保存/清除按钮	在 C 端本地保存模型配置，不上传到 S 端或 B 端。
本地 OpenCode 面板	OpenCode 地址、启动/进入对话/停止按钮、运行状态	启动 C 端本地 OpenCode, 展示健康状态和 KB Session 入口。
BearFRP 发布面板	本地 LLM-Wiki 地址、代理类型、子域名/端口、隧道状态	供 B 端创建和管理 LLM-Wiki API 公网入口。
远程知识库面板	API 地址、名称、Token、健康检查、项目列表	供 C 端添加 B 端分享的 LLM-Wiki API 并选择项目。
MCP 连接面板	MCP 名称、工具列表、连接/检测按钮、错误提示	注册 llm-wiki-* MCP 并检测 read_file、search 等工具。
远程知识库列表	已保存 API、可用状态、当前项目、连接按钮	展示已添加的远程知识库，并支持切换或删除。
系统日志与错误提示	OpenCode 日志、MCP 注册错误、frp 状态、远程 API 错误	让联调和验收时能定位端口、域名、配置和 API 错误。

界面跳转关系遵循“配置模型和服务状态、添加远程知识库、连接 MCP、进入 OpenCode 对话”的结构。发生 URL 不完整、通配域名未配置、远程 API 不可达、OpenCode 未就绪或 MCP 注册失败时，系统应在当前界面给出明确状态和可执行处理建议，而不是只输出底层错误日志。

3.3 用例设计

系统主要参与者包括 B 端知识拥有者、C 端知识消费者和 S 端管理员。用例关系如图 2 所示。

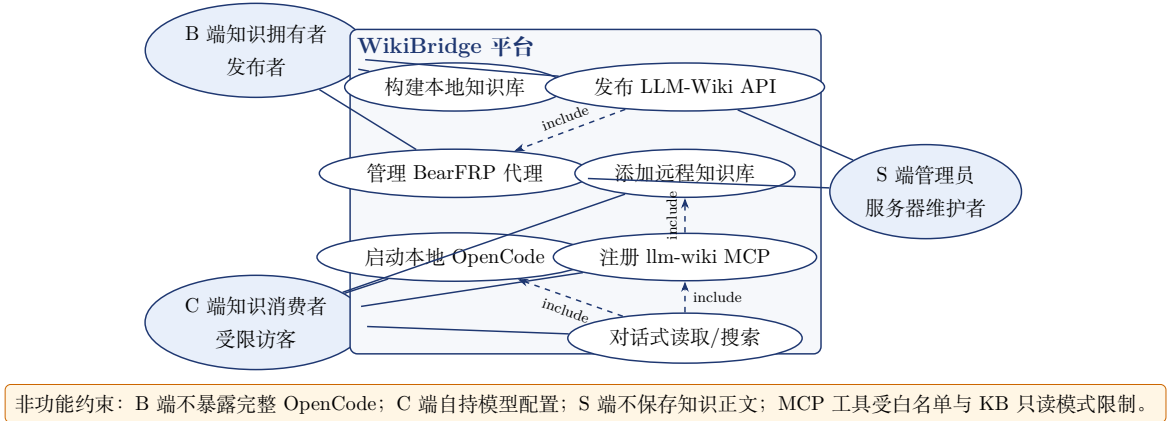


图 2: WikiBridge 用例图

3.3.1 构建本地知识库用例

B 端知识拥有者在本地准备资料并运行 LLM-Wiki，系统生成或维护本地 Wiki 项目、文件和索引。该用例由 LLM-Wiki 内部构建流程承担，WikiBridge Desktop 主要关注服务健康、项目列表和 API 是否可分享。

3.3.2 发布 LLM-Wiki API 用例

B 端用户登录 BearFRP 后创建 HTTP 或 TCP 代理。系统校验本地 LLM-Wiki API 可访问后启动 frpc，将本地 API 映射到 S 端公网入口。HTTP 子域名模式要求真实通配域名已经解析到 S 端；否则系统应提示配置域名或切换 TCP 模式。发布成功后，B 端获得可分享的 LLM-Wiki API URL。

3.3.3 添加远程知识库用例

C 端用户在 WikiBridge Desktop 中粘贴 B 端分享的 LLM-Wiki API URL，并可填写名称和 Token。系统调用健康检查和项目列表接口，确认 API 可达后保存远程知识库配置，并允许用户选择具体项目。

3.3.4 启动本地 OpenCode 与注册 MCP 用例

C 端用户保存模型配置后，Desktop 启动本地 OpenCode 并创建 KB Chat Session。随后系统注册 llm-wiki-* MCP，使 OpenCode 能调用 llm_wiki_read_file、llm_wiki_search 等受控工具访问远程 LLM-Wiki API。

3.3.5 对话式读取与搜索用例

C 端用户在本地的 OpenCode 中提问。OpenCode 根据模型推理和工具描述调用 llm-wiki MCP，MCP server 向远程 LLM-Wiki API 发起文件读取、项目列表或搜索请求。工具结果返回 OpenCode 后，由 C 端模型配置生成回答。该过程中 B 端不运行 C 端访客的 Agent 行为。

3.4 核心流程设计

系统主流程由 B 端 API 发布、C 端远程知识库添加、OpenCode 启动、MCP 注册和工具调用组成，如图 3 所示。

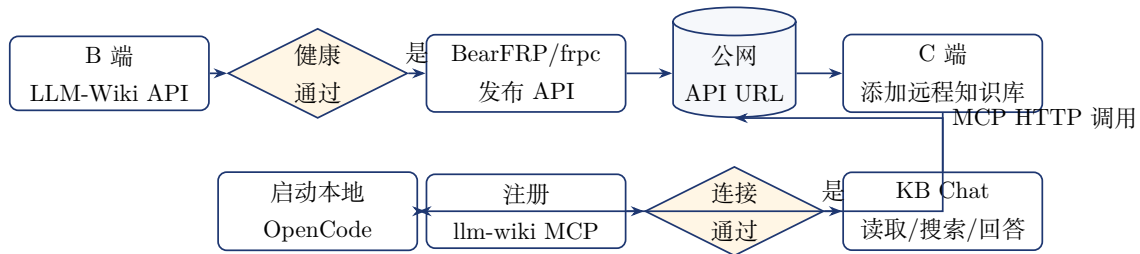


图 3: B 端 API 发布与 C 端 OpenCode/MCP 消费核心流程图

3.5 类设计和构件设计

当前阶段采用构件级设计结合关键类设计。构件划分如表 5 所示。

表 5: 主要构件职责

构件	关键类/对象	职责
桌面端壳层	DesktopShell	聚合 BearFRP、OpenCode、远程知识库和 MCP 状态，提供统一操作入口。
	StatusPanel	
	ErrorToast	
模型配置	ModelConfigStore	在 C 端本地保存模型供应商、模型名和 API Key，支持保存和清除。
	ProviderConfig	
	ApiKeyStore	
B 端发布控制	LlmWikiServiceRef	检查本地 LLM-Wiki API，创建 BearFRP 代理，启动和监控 frpc。
	FrpcController	
	TunnelStatusWatcher	
BearFRP 控制面	BearfrpClient	处理认证、代理创建、HTTP/TCP 发布入口、在线状态和脚本生成。
	ProxyService	
	FrpsStatusCollector	
C 端远程知识库	RemoteKnowledgeBase	保存远程 API URL 和 Token，执行健康检查、项目列表、文件读取和搜索。
	LlmWikiApiClient	
	ProjectSelector	

构件	关键类/对象	职责
OpenCode sidecar	OpenCodeSidecar	启动本地 OpenCode，创建 KB Chat Session，检测运行状态。
	KbSessionManager	
	OpenCodeHealthProbe	
MCP 注册与工具	McpRegistry	注册 llm-wiki-* MCP，限制工具名称、命令、入口文件和环境变量。
	LlmWikiMcpServer	
	ToolWhitelist	
KB 安全边界	KbGuard	在 OpenCode KB 模式下阻断危险文件访问、写入操作和不受控工具调用。
	ReadonlyPolicy	
	ToolPolicy	

3.5.1 关键类关系

系统类关系以端侧控制类协调任务、实体类保存状态、边界类展示界面为主。核心关系如下：

- 桌面端壳层聚合 OpenCode sidecar、远程知识库、MCP 注册和 frpc 控制器的状态，并把错误映射为用户可理解提示。
- 远程知识库对象持有远程 API URL、Token 和当前项目，依赖 LLM-Wiki API 客户端执行健康检查、项目列表、文件读取和搜索。
- OpenCodeSidecar 负责启动本地 OpenCode，并由 KbSessionManager 创建 KB Chat Session。
- MCP 注册器读取远程知识库配置，注册 llm-wiki MCP server，并通过工具白名单限制 llm-wiki-* 工具范围。
- frpc 控制器调用 BearFRP 客户端获取代理配置，再启动 B 端 frpc 进程；隧道状态监听器负责在线状态和重连提示。
- KbGuard 约束 OpenCode KB 模式下的文件访问、写入和工具调用策略。

3.5.2 关键方法设计

类	方法	说明
ModelConfigStore	save(provider, model, apiKey)	在 C 端安全保存模型配置，不上传到 S 端。

LlmWikiApiClient	<code>health(baseUrl); listProjects()</code>	检查远程 LLM-Wiki API 可用性并读取项目列表。
Remote KnowledgeBase	<code>connect(); selectProject(id)</code>	保存 API URL、Token 和当前项目，并维护可用状态。
OpenCodeSidecar	<code>start() / stop() / health()</code>	启动或停止本地 OpenCode，并返回运行状态。
KbSessionManager	<code>createKbSession(remoteKb)</code>	为远程知识库创建 OpenCode KB Chat Session。
McpRegistry	<code>registerLlmWiki(remoteKb)</code>	注册 llm-wiki-* MCP，并写入必要环境变量。
LlmWikiMcpServer	<code>readFile(project, path); search(query)</code>	将 OpenCode 工具调用转换为远程 LLM-Wiki API 请求。
FrpcController	<code>publishApi(localPort)</code>	B 端申请公网入口、生成配置、启动 frpc 并返回 API URL。
BearfrpClient	<code>createProxy(type, localPort)</code>	调用 BearFRP 控制面创建 HTTP/TCP 代理。
KbGuard	<code>enforceReadonly(toolCall)</code>	在 KB 模式下阻断写入、删除、终端执行和非白名单 MCP。

3.6 数据设计

WikiBridge 的数据设计分为 B 端知识库数据、S 端控制面数据和 C 端本地配置数据。系统明确避免把用户知识内容上传到 S 端持久化保存。

3.6.1 B 端数据结构

路径/文件	数据内容	说明
<code>/raw</code>	原始资料	B 端用户本地保存的 Word、PDF、网页摘录、笔记、Markdown、TXT 等。
<code>/wiki</code>	Wiki 页面	LLM-Wiki 生成或维护的结构化页面和元数据。

<code>/index</code>	本地索引	供 LLM-Wiki 搜索和 API 查询使用的索引。
<code>llm-wiki config</code>	服务配置	B 端 LLM-Wiki 服务端口、项目目录、模型或构建配置。
<code>frpc config</code>	发布配置	B 端 frpc 连接 S 端 frps 的 token、协议、子域名或端口。
<code>publish cache</code>	发布状态缓存	当前公网 API URL、隧道状态、最近连接时间和可回收标识。

3.6.2 C 端本地配置数据

数据项	说明
<code>ModelConfig</code>	模型供应商、模型名和 API Key 保存位置。API Key 应使用系统安全存储能力或加密保存。
<code>Remote KnowledgeBase</code>	远程 LLM-Wiki API URL、名称、Token、当前项目、健康检查状态和最近检测时间。
<code>OpenCodeState</code>	本地 OpenCode 地址、端口、运行状态、当前 KB Session。
<code>McpRegistration</code>	MCP 名称、命令、入口文件、环境变量、工具列表和注册状态。
<code>UserPreference</code>	默认模型、默认远程知识库、日志级别和界面偏好。

3.6.3 S 端控制面数据表

S 端只保存平台控制面数据。建议设计以下数据表：

数据表	字段说明
<code>T_User</code>	<code>id</code> 、 <code>account</code> 、 <code>password_hash</code> 、 <code>display_name</code> 、 <code>status</code> 、 <code>created_at</code> 、 <code>last_login_at</code> 。用于保存平台账号，不保存知识内容。
<code>T_AccessToken</code>	<code>id</code> 、 <code>user_id</code> 、 <code>token_hash</code> 、 <code>expire_at</code> 、 <code>revoked_at</code> 。用于 Token 校验和吊销。
<code>T_Proxy</code>	<code>id</code> 、 <code>user_id</code> 、 <code>type</code> 、 <code>subdomain</code> 、 <code>public_port</code> 、 <code>local_port</code> 、 <code>status</code> 、 <code>created_at</code> 、 <code>closed_at</code> 。用于记录 HTTP/TCP 发布入口。
<code>T_TunnelStatus</code>	<code>id</code> 、 <code>proxy_id</code> 、 <code>online</code> 、 <code>bandwidth_in</code> 、 <code>bandwidth_out</code> 、 <code>connection_count</code> 、 <code>last_heartbeat</code> 。用于管理 frp 状态。

`T_RateLimitPolicy` `id`、`user_id`、`max_bandwidth`、`max_connections`、`enabled`。用于基础限速和防滥用。

3.6.4 数据安全设计

本地配置中的 Token 和 API Key 应加密保存或使用系统安全存储能力。日志中不得输出完整 Token、API Key、用户密码、原始资料全文和模型请求完整上下文。S 端密码只保存哈希值，Token 只保存哈希或可验证摘要。发布入口应支持用户主动关闭，管理员也可在滥用或异常流量场景下回收入口。B 端不得将完整 OpenCode 暴露到公网，C 端不得将模型 API Key 上传给 B 端或 S 端。

3.7 部署设计

系统采用 B/S/C 分布式部署方式，部署关系如图 4 所示。

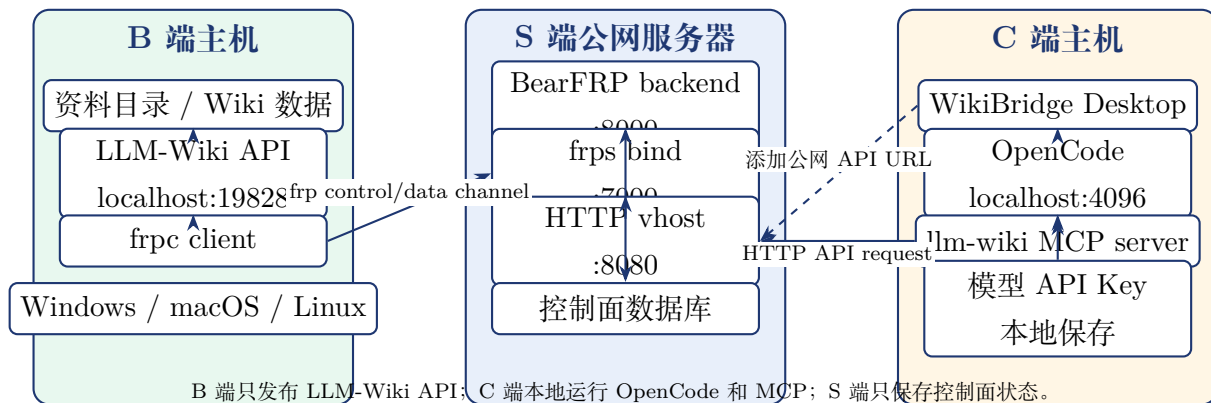


图 4: WikiBridge B/S/C 部署图

3.7.1 B 端部署

B 端部署 LLM-Wiki API 和 frpc。LLM-Wiki API 默认监听本地端口，例如 19828，frpc 只将该 API 发布到 S 端公网入口。B 端不部署对外开放的 OpenCode 页面，也不向 C 端共享模型 API Key。

3.7.2 S 端部署

S 端部署在具有公网 IP 或公网域名的 Linux 主机上，包含 BearFRP backend、frps、HTTP vhost 和控制面数据库。HTTP 子域名模式需要将通配域名解析到 S 端；若不具备域名条件，应使用 TCP 端口模式作为替代。

3.7.3 C 端部署

C 端部署 WikiBridge Desktop、OpenCode 和 llm-wiki MCP server。OpenCode 默认监听本地端口，例如 4096。C 端用户保存自己的模型配置，添加远程知识库 API URL 后，

通过 MCP 工具访问 B 端 LLM-Wiki API。

3.8 异常处理与可靠性设计

异常场景	检测方式	处理策略
LLM-Wiki API 不可达	B/C 端健康检查失败或连接超时	提示检查 B 端 LLM-Wiki 服务、端口和防火墙，不创建远程知识库。
HTTP 子域名不可解析	C 端访问公网 URL 失败或域名解析异常	提示配置真实通配域名，或切换 TCP 发布模式。
Token 过期或无权限	远程 API 返回 401/403	提示重新登录、更新 token 或联系 B 端发布者。
隧道断开	frpc 心跳或 frps 状态异常	自动重连并在界面展示状态；多次失败后提示检查网络和 S 端配置。
OpenCode 未就绪	本地 4096 健康检查失败	提示启动 OpenCode、检查端口占用或查看 sidecar 日志。
MCP 注册失败	OpenCode MCP 列表缺少 llm-wiki-* 或工具调用失败	回滚注册状态，提示检查 MCP 命令、入口文件、环境变量和远程 URL。
模型 API 配置缺失	OpenCode 对话返回模型配置错误	提示在 C 端模型设置中保存 provider、model 和 API Key。
KB 安全策略命中	工具调用请求写入、删除或非白名单 MCP	拒绝执行并记录审计日志，提示当前为 KB 只读模式。

3.9 测试设计

课程设计原型阶段建议按以下场景验收：

1. B 端 API 测试：启动 LLM-Wiki API 后，确认 /api/v1/health、项目列表、文件读取和搜索接口可用。
2. BearFRP 发布测试：创建 HTTP/TCP 代理，确认 frpc 连接 frps，公网 API URL 可访问。
3. 通配域名测试：HTTP 子域名模式下验证域名解析；缺少通配域名时确认系统给出明确错误或降级建议。

4. C 端远程知识库测试：粘贴公网 API URL 后，确认 Desktop 能完成健康检查、保存远程知识库并列出项目。
5. OpenCode 启动测试：确认 C 端可启动本地 OpenCode，创建 KB Chat Session。
6. MCP 注册测试：确认 llm-wiki-* MCP 出现在 OpenCode 可用工具列表，并能调用 `read_file` 和 `search`。
7. 安全边界测试：确认 B 端未暴露完整 OpenCode，C 端模型 Key 不上传到 S 端，KB 只读模式和 MCP 白名单生效。
8. 异常测试：模拟远程 API 不可达、Token 错误、OpenCode 未就绪、MCP 注册失败和模型配置缺失，确认界面提示清晰且系统可恢复。

结论

WikiBridge 的设计目标是验证 “B 端 LLM-Wiki API 发布 + S 端 BearFRP/frps 转发 + C 端 OpenCode/MCP 消费” 的端到端可行性。系统通过 B/S/C 职责分离和 Agent 能力本地化，避免把 B 端完整 OpenCode 暴露到公网，也避免把 C 端模型 Key 交给 B 端或 S 端。后续实现应优先保证 B 端 API 发布、C 端远程知识库添加、OpenCode KB Session 和 llm-wiki MCP 工具调用闭环，再逐步完善 token 鉴权、限速策略、跨平台打包和自动化测试。